

torch.func.jvp#

<https://pytorch.org/docs/stable/generated/torch.func.jvp.html>

Description:

Standing for the Jacobian-vector product, returns a tuple containing the output of `func(*primals)` and the “Jacobian of `func` evaluated at `primals`” times `tangents`. This is also known as forward-mode autodiff.

`func` (function) – A Python function that takes one or more arguments, one of which must be a Tensor, and returns one or more Tensors

`primals` (Tensors) – Positional arguments to `func` that must all be Tensors. The returned function will also be computing the derivative with respect to these arguments

`tangents` (Tensors) – The “vector” for which Jacobian-vector-product is computed. Must be the same structure and sizes as the inputs to `func`.

`has_aux` (bool) – Flag indicating that `func` returns a `(output, aux)` tuple where the first element is the output of the function to be differentiated and the second element is other auxiliary objects that will not be differentiated. Default: False.

Function Signature

```
torch.func.jvp(func, primals, tangents, *, strict=False,  
has_aux=False)[source]#
```

Parameters

Returns

Returns:

Returns a (output, jvp_out) tuple containing the output of func evaluated at primals and the Jacobian-vector product. If has_aux is True, then instead returns a (output, jvp_out, aux) tuple.

Code Examples

```
>>> from torch.func import jvp
>>> x = torch.randn(3)
>>> f = lambda x: x * torch.tensor([1.0, 2.0, 3])
>>> value, grad = jvp(f, (x,), (torch.tensor(1.0),))
>>> assert torch.allclose(value, f(x))
>>> assert torch.allclose(grad, torch.tensor([1.0, 2, 3]))
```

```
>>> from torch.func import jvp
>>> x = torch.randn(5)
>>> y = torch.randn(5)
>>> f = lambda x, y: (x * y)
>>> _, output = jvp(f, (x, y), (torch.ones(5), torch.ones(5)))
>>> assert torch.allclose(output, x + y)
```

Notes & Warnings

NOTE

You may see this API error out with “forward-mode AD not implemented for operator X”. If so, please file a bug report and we will prioritize it.

Detailed Documentation

Documentation

Standing for the Jacobian-vector product, returns a tuple containing the output of `func(*primals)` and the “Jacobian of `func` evaluated at `primals`” times tangents. This is also known as forward-mode autodiff.

`func` (function) – A Python function that takes one or more arguments, one of which must be a Tensor, and returns one or more Tensors

`primals` (Tensors) – Positional arguments to `func` that must all be Tensors. The returned function will also be computing the derivative with respect to these arguments

`tangents` (Tensors) – The “vector” for which Jacobian-vector-product is computed. Must be the same structure and sizes as the inputs to `func`.

`has_aux` (bool) – Flag indicating that `func` returns a (output, aux) tuple where the first element is the output of the function to be differentiated and the second element is other auxiliary objects that will not be differentiated. Default: False.

Returns a (output, `jvp_out`) tuple containing the output of `func` evaluated at `primals` and the Jacobian-vector product. If `has_aux` is True, then instead returns a (output, `jvp_out`, aux) tuple.

You may see this API error out with “forward-mode AD not implemented for operator X”. If so, please file a bug report and we will prioritize it.

`jvp` is useful when you wish to compute gradients of a function $R^1 \rightarrow R^N$

`jvp()` can support functions with multiple inputs by passing in the tangents for each of the inputs
